

ΠΑΝΕΠΙΣΤΗΜΙΟ ΘΕΣΣΑΛΙΑΣ

ΠΟΛΥΤΕΧΝΙΚΗ ΣΧΟΛΗ

ΤΜΗΜΑ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ

ΕΦΑΡΜΟΓΗ ΟΛΟΚΛΗΡΩΜΕΝΗΣ ΒΔ ΣΕ DBMS ΤΗΣ ΕΠΙΛΟΓΗΣ ΜΑΣ

ΣΥΣΤΗΜΑΤΑ ΒΑΣΕΩΝ ΔΕΔΟΜΕΝΩΝ ΙΙ - ΕΡΓΑΣΙΑ ΕΞΑΜΗΝΟΥ

Γεώργιος Μαστέλλος - 03402
Γεώργιος Μοιρώτσος - 03401

Επιβλέπουσα Καθηγήτρια: Τουσίδου Ελένη

Βόλος 2025

Περιεχόμενα

1 ΕΙΣΑΓΩΓΗ **2**

2 ΠΕΡΙΓΡΑΦΗ ΣΧΕΣΙΑΚΟΥ ΣΧΗΜΑΤΟΣ **3**
2.1 Συνολική θεώρηση 3
2.2 Πίνακας airport 4
2.3 Πίνακας flight 4
2.4 Πίνακας airline 5
2.5 Πίνακας passengers 6
2.6 Πίνακας aircraft 7

3 Οδηγίες Εγκατάστασης και Χρήσης **9**
3.1 Εγκατάσταση 9
3.1.1 Εγκατάσταση Python και απαιτούμενων βιβλιοθηκών . . 9
3.2 Εισαγωγή της βάσης δεδομένων μέσω Command Line 9
3.3 Αρχιτεκτονική Εφαρμογής 10
3.4 Τρόπος εκτέλεσης της εφαρμογής 13
3.5 Οδηγίες περιήγησης στη σελίδα "Aerofind" 15

4 Σχολιασμός κώδικα **20**
4.1 Υλοποίηση Προβολής Πινάκων (View Tables) 20
4.2 Εισαγωγή Νέας Πτήσης 20
4.3 Διαγραφή Πτήσης 21
4.4 Αναζήτηση Πτήσεων με Φίλτρα 21
4.5 Αναζήτηση Επιβατών και Υπολογισμός Διαθέσιμων Θέσεων . . . 22

5 Δεδομένα που χρησιμοποιήθηκαν **23**
5.1 Είδη δεδομένων ανά πίνακα 23
5.2 Ποσότητα και κάλυψη δεδομένων 24
5.3 Περιορισμοί και παραδοχές 25
5.4 Αυτοματοποιημένη Εισαγωγή Δεδομένων με Python Scripts . . . 25

6 Συμπεράσματα και μελλοντική χρήση **27**

1 ΕΙΣΑΓΩΓΗ

Η συγκεκριμένη αναφορά αποτελεί εργασία εξαμήνου για το μάθημα "Συστήματα Βάσεων Δεδομένων ΙΙ". Η εργασία είναι προγραμματιστική. Η παρούσα αναφορά έχει ως στόχο την επεξήγηση της βάσης δεδομένων (ΒΔ) που αναπτύξαμε και χρησιμοποιήσαμε, οδηγίες εγκατάστασης και χρήσης της πλατφόρμας που σχεδιάσαμε, καθώς και σχολιασμός του κώδικα που αναπτύχθηκε τόσο για την δημιουργία της ΒΔ με τη χρήση της MySQL όσο και για τη σύνδεση της ΒΔ με την εφαρμογή, χρησιμοποιώντας τη γλώσσα Python.

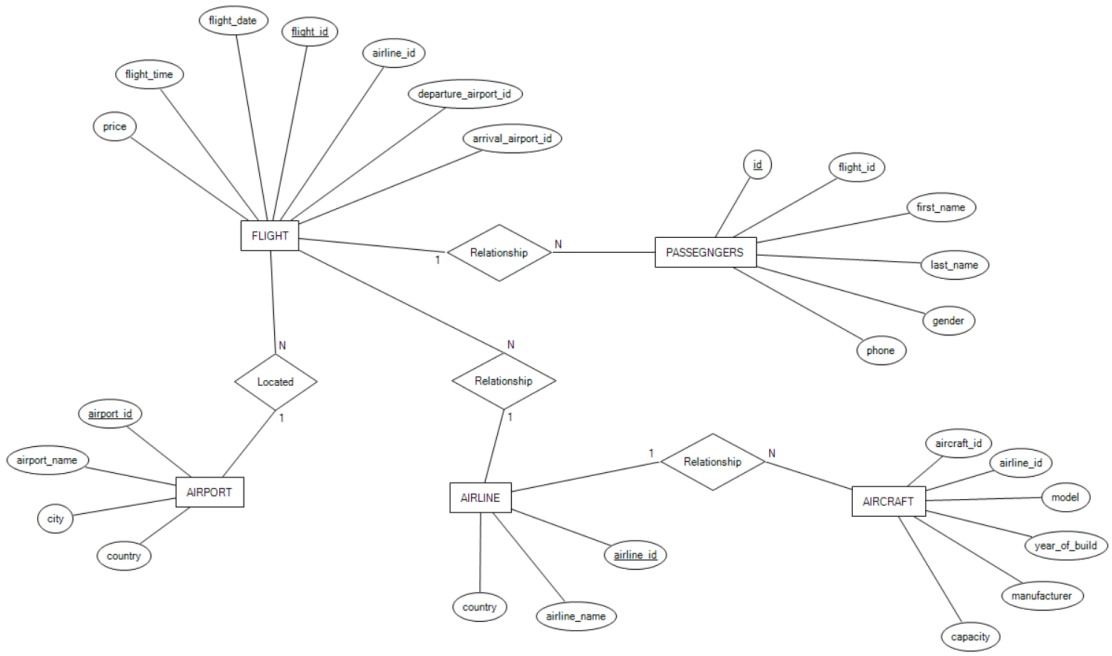
Η εφαρμογή παρουσιάζει ένα ολοκληρωμένο σύστημα διαχείρισης πτήσεων, το οποίο λειτουργεί ως διαδραστική μηχανή αναζήτησης για αεροπορικά ταξίδια ανά τον κόσμο. Ο χρήστης μπορεί εύκολα να εισάγει, να αναζητήσει, να διαγράψει ή να προβάλλει δεδομένα πτήσεων μέσα από ένα φιλικό προς τον χρήστη περιβάλλον. Τα δεδομένα των πινάκων έχουν προσαρμοστεί για τους σκοπούς της εργασίας. Για τον λόγο αυτό προτείνονται παρακάτω συγκεκριμένα βήματα που πρέπει να ακολουθήσει ένας χρήστης ως προς την περιήγησή του στη σελίδα.

Κύριος σκοπός της παρούσας προγραμματιστικής εργασίας είναι ο φοιτητής να αποκτήσει μια ολοκληρωμένη εικόνα τόσο για τον σχεδιασμό μιας ολοκληρωμένης ΒΔ όσο και για τη σύνδεσή της με ένα frontend περιβάλλον, με τη χρήση της γλώσσας Python.

Η **Ενότητα 2** παρέχει όλες τις απαραίτητες πληροφορίες ως προς το σχεσιακό σχήμα της ΒΔ που δημιουργήσαμε και ως προς τους πινάκες και τις στήλες τους. Η **Ενότητα 3** περιλαμβάνει οδηγίες εγκατάστασης για την MySQL, την γλώσσα Python, καθώς και τις απαραίτητες βιβλιοθήκες. Περιγράφεται, επίσης, η αρχιτεκτονική της εφαρμογής. Ιδιαίτερα σημαντικές είναι οι **υπο-ενότητες 3.4 και 3.5**, διότι αναφέρονται οδηγίες τόσο για το άνοιγμα της εφαρμογής όσο και την περιήγηση του χρήστη μέσα στην εφαρμογή. Η **Ενότητα 4** με τη σειρά της αναλύει τον κώδικα που αναπτύξαμε στο backend επίπεδο, για τη σύνδεση της MySQL με την εφαρμογή χρησιμοποιώντας τη γλώσσα python.Στην **Ενότητα 5** περιγράφουμε τα δεδομένα που χρησιμοποιήθηκαν σε κάθε πίνακα. Τέλος, η **Ενότητα 6** καταγράφει τα συμπεράσματα και τη μελλοντική χρήση / βελτίωση.

2 ΠΕΡΙΓΡΑΦΗ ΣΧΕΣΙΑΚΟΥ ΣΧΗΜΑΤΟΣ

2.1 Συνολική θεώρηση



Σχήμα 1: Σχεσιακό Σχήμα

Πίνακας 1: Σχέσεις και ξένα κλειδιά μεταξύ των πινάκων

Γονικός πίνακας	**Τέκνο πίνακας**	**Πληθικότητα**	
AIRLINE(airline_id)	FLIGHT(airline_id)	1 : N	
AIRPORT(airport_id)	FLIGHT(departure_airport_id)	1 : N	
AIRPORT(airport_id)	FLIGHT(arrival_airport_id)	1 : N	
FLIGHT(id)	PASSENGERS(flight_id)	1 : N	
AIRLINE(airline_id)	AIRCRAFT(airline_id)	1 : N	

2.2 Πίνακας airport

Ο πίνακας <<airport>> αποτελείται από 4 στήλες: **airport_id, airport_name, city, country.**

- **airport_id:** Πρωτεύον κλειδί που δηλώνει το μοναδικό id (κωδικό) κάθε αεροδρομίου.
- **airport_name:** Δηλώνει το όνομα του αεροδρομίου
- **city:** Η πόλη που βρίσκεται το αεροδρόμιο
- **country:** Η χώρα που βρίσκεται το αεροδρόμιο

Έχουν εισαχθεί συνολικά 35 αεροδρόμια. Ενδεικτικά, ο πίνακας έχει την παρακάτω μορφή:

airport_id	airport_name	city	country
1	Athens International Airport	Athens	Greece
2	Heathrow Airport	London	United Kingdom
3	Charles de Gaulle Airport	Paris	France
4	Frankfurt Airport	Frankfurt	Germany
5	John F. Kennedy International Airport	New York	USA

Σχήμα 2: Πίνακας airport

2.3 Πίνακας flight

Ο πίνακας <<flight>> αποτελείται από 7 στήλες: **id, airline_id, departure_airport_id, airline_airport_id, flight_date, flight_time, price.**

- **id :** Πρωτεύον κλειδί που δηλώνει το μοναδικό id κάθε πτήσης.
- **airline_id :** Ξένο κλειδί ως προς το airline_id του πίνακα <<airline>>. Δηλώνει σε ποια αεροπορική εταιρεία ανήκει η πτήση.
- **departure_airport_id :** Ξένο κλειδί ως προς το airport_id του πίνακα <<airport>>. Δηλώνει από ποιο αεροδρόμιο αναχωρεί η πτήση.
- **arrival_airport_id :** Ξένο κλειδί ως προς το airport_id του πίνακα <<airport>>. Δηλώνει σε ποιο αεροδρόμιο προσγειώνεται η πτήση.

- **flight_date :** Ημερομηνία πτήσης.
- **flight_time :** Ώρα πτήσης.
- **price :** Τιμή πτήσης.

Οι ονομασίες των πόλεων αναχώρησης / άφιξης συμπεριλαμβάνονται στον πίνακα <<airport>> (λαμβάνονται με JOIN πινάκων). Ενδεικτικά, ο πίνακας έχει την παρακάτω μορφή:

id	airline_id	departure_airport_id	arrival_airport_id	flight_date	flight_time	price
68	20	12	19	2026-03-05	11:05:00	145.00
69	20	12	19	2026-03-06	12:30:00	260.00
70	20	12	19	2026-03-07	13:10:00	88.00
71	20	12	19	2026-03-08	13:55:00	299.00
72	20	12	19	2026-03-09	14:35:00	170.00
73	20	12	19	2026-03-10	15:50:00	130.50

Σχήμα 3: Πίνακας flight

2.4 Πίνακας airline

Ο πίνακας <<airline>> αποτελείται από 3 στήλες: **airline_id, airline_name, country.**

- **airline_id:** Πρωτεύον κλειδί που δηλώνει το μοναδικό id (κωδικό) κάθε αεροπορίας.
- **airline_name:** Όνομα αεροπορικής εταιρείας
- **country:** Χώρα προέλευσης αεροπορικής εταιρείας

Έχουν εισαχθεί συνολικά 35 αεροπορικές εταιρείες. Ενδεικτικά, ο πίνακας έχει την παρακάτω μορφή:

airline_id	airline_name	country
1	Aegean Airlines	Greece
2	British Airways	United Kingdom
3	Air France	France
4	Lufthansa	Germany
5	Emirates	United Arab Emirates
6	Singapore Airlines	Singapore
7	American Airlines	USA

Σχήμα 4: Πίνακας airline

2.5 Πίνακας passengers

Ο πίνακας <<passengers>> αποτελείται από 6 στήλες: **id, flight_id, gender, first_name, last_name, phone.**

- **id :** Πρωτεύον κλειδί που δηλώνει το μοναδικό id κάθε επιβάτη (τύπου ΑΜΚΑ).
- **flight_id :** Ξένο κλειδί ως προς το id της πτήσης του πίνακα <<flight>>. Δηλώνει σε ποια πτήση βρίσκεται ο επιβάτης.
- **gender :** Φύλλο του κάθε επιβάτη (M : male, F : female).
- **first_name :** Μικρό όνομα κάθε επιβάτη.
- **last_name :** Επώνυμο επιβάτη.
- **phone :** Τηλέφωνο επικοινωνίας του επιβάτη.

Ενδεικτικά, ο πίνακας έχει την παρακάτω μορφή:

id	flight_id	gender	first_name	last_name	phone
200	99	F	Sofia	Martinez	6971000004
201	99	M	Nikos	Georgiou	6971000005
202	99	M	Daniel	Brown	6971000006
203	99	F	Elena	Kosta	6971000007
204	99	M	Victor	Sanchez	6971000008
205	99	M	Jason	Mitchell	6971000009
206	99	M	George	Manolis	6971000010
207	99	F	Anna	Rivera	6971000011

Σχήμα 5: Πίνακας passengers

2.6 Πίνακας aircraft

Ο πίνακας <<aircraft>> αποτελείται από 9 στήλες: **aircraft_id, airline_id, capacity, tail_number, manufacturer, model, year_of_build, status, base_airport_id**

- **aircraft_id :** Πρωτεύον κλειδί που δηλώνει το μοναδικό id κάθε αεροσκάφους.
- **airline_id :** Ξένο κλειδί ως προς το airline_id του πίνακα <<airline>>. Δηλώνει σε ποια αεροπορική εταιρεία ανήκει το αεροσκάφος.
- **capacity :** Χωρητικότητα αεροσκάφους ως προς τους επιβάτες.
- **manufacturer :** Κατασκευαστής (εταιρεία).
- **model :** Μοντέλο αεροσκάφους.
- **year_of_build :** Χρονιά που κατασκευάστηκε το αεροσκάφος.
- **status :** Κατάσταση αεροσκάφους (π.χ. active, retired).

Ενδεικτικά, ο πίνακας έχει την παρακάτω μορφή:

aircraft_id	airline_id	capacity	manufacturer	model	year_of_build	status
1	1	180	Airbus	A320-200	2015	active
2	2	350	Boeing	747-400	2000	retired
3	3	144	Airbus	A319-100	2011	active
4	4	200	Airbus	A321-100	2016	active
5	5	517	Airbus	A380-800	2018	active
6	6	318	Boeing	787-10	2020	active
7	7	172	Boeing	737-800	2013	active

Σχήμα 6: Πίνακας aircraft

3 Οδηγίες Εγκατάστασης και Χρήσης

3.1 Εγκατάσταση

Η εφαρμογή Aerofind υλοποιήθηκε χρησιμοποιώντας Python (Flask framework) και σύστημα βάσης δεδομένων MySQL. Για την ορθή λειτουργία του συστήματος απαιτείται η εγκατάσταση του Python περιβάλλοντος, των απαραίτητων βιβλιοθηκών, καθώς και η δημιουργία της βάσης δεδομένων uni_app με τα αντίστοιχα δεδομένα.

3.1.1 Εγκατάσταση Python και απαιτούμενων βιβλιοθηκών

Απαιτείται Python 3.9 ή νεότερη έκδοση.

1. Κατεβάζουμε και εγκαθιστούμε Python από τη διεύθυνση: `https://www.python.org/downloads/`
2. Στη συνέχεια, εγκαθιστούμε τις βιβλιοθήκες που χρησιμοποιούνται στην εφαρμογή:

```
pip install flask
pip install mysql-connector-python
```

3.2 Εισαγωγή της βάσης δεδομένων μέσω Command Line

Η βάση δεδομένων μπορεί να εισαχθεί μέσω Command Line (Command Prompt ή PowerShell).

Βήμα 1: Άνοιγμα τερματικού

Ο χρήστης ανοίγει το Command Prompt ή το PowerShell στα Windows.

Βήμα 2: Μετάβαση στον φάκελο του αρχείου `.sql`

Με τη χρήση της εντολής `cd`, ο χρήστης μεταβαίνει στον φάκελο που περιέχει το αρχείο της βάσης δεδομένων:

```
cd "C:\path\to\folder"
```

Βήμα 3: Εισαγωγή δεδομένων σε υπάρχουσα βάση

Αν η βάση δεδομένων `uni_app` υπάρχει ήδη, η εισαγωγή των δεδομένων πραγματοποιείται με μία εντολή:

```
"C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" -u root
   -p uni_app < uni_app_database.sql
```

Συνήθως το κομμάτι στις "" είναι το παραπάνω. Σε αντίθετη περίπτωση ψάχνουμε που είναι αποθηκευμένο το mysql.exe.. Αντίστοιχα, αν είμαστε στο Βήμα 4 και 5 μπορεί πάλι να χρειαστεί το directory ολόκληρο. Στη συνέχεια ζητείται ο κωδικός πρόσβασης του χρήστη `root`(**1234**). Μετά την εισαγωγή, το script εκτελείται αυτόματα και η βάση γεμίζει με τα δεδομένα.

Βήμα 4: Δημιουργία βάσης (αν δεν υπάρχει)

Σε περίπτωση που η βάση `uni_app` δεν υπάρχει, ο χρήστης συνδέεται πρώτα στον MySQL server:

```
    mysql -u root -p
```

Στο prompt της MySQL εκτελείται η δημιουργία της βάσης:

```
CREATE DATABASE uni_app
CHARACTER SET utf8mb4
COLLATE utf8mb4_general_ci;
EXIT;
```

Βήμα 5: Επαναφορά δεδομένων

Μετά τη δημιουργία της βάσης, επαναλαμβάνεται η εισαγωγή του αρχείου:

```
    mysql -u root -p uni_app < uni_app_database.sql
```

Με την ολοκλήρωση της διαδικασίας, η βάση δεδομένων είναι έτοιμη προς χρήση από την εφαρμογή Flask.

3.3 Αρχιτεκτονική Εφαρμογής

Η εφαρμογή που αναπτύχθηκε υλοποιεί ένα διαδραστικό σύστημα διαχείρισης αεροπορικών δεδομένων, το οποίο βασίζεται σε αρχιτεκτονική τύπου πελάτη–εξυπηρετητή. Το backend υλοποιείται με Python και Flask, ενώ το frontend αποτελείται από ένα ενιαίο αρχείο HTML που φορτώνεται από το Flask και περιλαμβάνει ενσωματωμένη JavaScript και CSS. Η βάση δεδομένων είναι υλοποιημένη

σε MySQL και εξυπηρετεί όλα τα αιτήματα ανάκτησης, εισαγωγής, αναζήτησης και διαγραφής δεδομένων.

Δομή Αρχείων

Η εφαρμογή αποτελείται από δύο βασικά αρχεία:

- **new_app.py**: ο κεντρικός server της εφαρμογής, ο οποίος διαχειρίζεται τη σύνδεση με την MySQL, τα routes του Flask και την εκτέλεση SQL ερωτημάτων.
- **new_app.html**: το κεντρικό front-end αρχείο, το οποίο περιλαμβάνει το περιβάλλον χρήστη, τα κουτιά επιλογών και την JavaScript που επικοινωνεί με τα endpoints του Flask.

Backend (Flask)

Το backend υλοποιείται με χρήση του `Flask`, και περιλαμβάνει λειτουργίες για:

- ασφαλή σύνδεση με τη βάση μέσω `mysql.connector`,
- εκτέλεση SQL εντολών (SELECT, INSERT, DELETE),
- μετατροπή τύπων όπως `date`, `time`, `datetime`, `timedelta` σε strings ώστε να μπορούν να επιστραφούν ως JSON,
- rendering του `new_app.html`.

Η σύνδεση με τη βάση γίνεται μέσω της συνάρτησης:

```
def get_connection():
    return mysql.connector.connect(
        host="localhost",
        user="root",
        password="1234",
        database="uni_app"
```

Όλα τα routes που απαιτούν πρόσβαση στη MySQL χρησιμοποιούν αυτή τη σύνδεση.

Frontend (HTML, CSS, JavaScript)

Το `new_app.html` περιλαμβάνει:

- responsive σχεδίαση με σύγχρονο UI,
- dark-mode αισθητική,
- λειτουργικές ενότητες (κουτιά) για:
  - προβολή πίνακα,
  - εισαγωγή πτήσης,
  - διαγραφή πτήσης,
  - αναζήτηση πτήσεων,
  - εύρεση καλύτερων προσφορών,
  - εύρεση επιβατών για συγκεκριμένη πτήση.

Ο χρήστης αλληλεπιδρά αποκλειστικά με το `new_app.html`, ενώ όλες οι λειτουργίες πραγματοποιούνται χωρίς ανανέωση της σελίδας.

Endpoints που αναπτύχθηκαν

Το backend περιλαμβάνει τα ακόλουθα routes:

- / — φόρτωση της εφαρμογής και ανάκτηση μοναδικών πόλεων.
- **/view_table** — επιστροφή όλων των γραμμών από επιτρεπόμενο πίνακα.
- **/insert_flight** — εισαγωγή νέας πτήσης με όλα τα στοιχεία.
- **/delete_flight** — διαγραφή πτήσης βάσει ID.
- **/search_flights** — αναζήτηση πτήσεων βάσει πόλης, προορισμού και ημερομηνίας (±2 ημέρες).
- **/search_best_offers** — εύρεση φθηνότερων πτήσεων ανά μήνα ανά πόλη.
- **/search_passengers** — εμφάνιση επιβατών συγκεκριμένης πτήσης, με υπολογισμό χωρητικότητας και εναπομεινάντων θέσεων.

Κάθε endpoint επιστρέφει JSON, κατάλληλο για επεξεργασία από JavaScript.

Λειτουργική Ροή Εφαρμογής

Η ροή λειτουργίας της εφαρμογής έχει ως εξής:

1. Ο χρήστης επιλέγει ενότητα από τα κουτιά του interface.
2. Το front-end στέλνει AJAX αίτημα στο Flask endpoint.
3. Το Flask εκτελεί το κατάλληλο SQL query μέσω MySQL.
4. Το αποτέλεσμα μετατρέπεται σε JSON και επιστρέφεται στο front-end.
5. Η JavaScript ενημερώνει δυναμικά τη σελίδα με τα αποτελέσματα.

Αυτή η αρχιτεκτονική προσφέρει:

- καθαρό διαχωρισμό backend–frontend,
- άμεση ανταπόκριση χωρίς επαναφόρτωση σελίδας,
- ευκολία συντήρησης,
- δυνατότητα μελλοντικής επέκτασης σε REST API ή desktop/mobile εφαρμογή.

3.4 Τρόπος εκτέλεσης της εφαρμογής

Για να εκτελέσουμε την web εφαρμογή `Aerofind` με χρήση Flask, ακολουθούμε τα παρακάτω βήματα.

Δομή φακέλων

Όλα τα αρχεία της εφαρμογής βρίσκονται σε έναν κοινό φάκελο εργασίας. Στο ανώτερο επίπεδο υπάρχει το αρχείο της εφαρμογής σε Python (π.χ. `app.py`) και ένας φάκελος `templates` που περιέχει το αρχείο HTML:

```
project_folder/
  uni_app_database.sql
  new_app.py
  templates/
    new_app.html
```

Το αρχείο `uni_app_database.sql` περιέχει την βάση δεδομένων που έχουμε δημιουργήσει. Το αρχείο `new_app.py` περιέχει τον κώδικα της Flask εφαρμογής και το αρχείο `new_app.html` αποτελεί το βασικό template της διεπαφής χρήστη.

Εκκίνηση της εφαρμογής από τη γραμμή εντολών

Για να τρέξουμε την εφαρμογή:

1. Ανοίγουμε τον φάκελο που περιέχει το `new_app.py` (π.χ. `project_folder/`) μέσα από την Εξερεύνηση αρχείων.
2. Στη γραμμή διευθύνσεων του παραθύρου γράφουμε `cmd` και πατάμε `Enter`, ώστε να ανοίξει το `Command Prompt` με τρέχον φάκελο τον φάκελο του `new_app.py`.
3. Στο `Command Prompt` εκτελούμε την εντολή:

Listing 1: Εκκίνηση της Flask εφαρμογής

```
python app.py
```

Αν όλα έχουν ρυθμιστεί σωστά (Python, Flask και σύνδεση με τη βάση δεδομένων), στο τερματικό εμφανίζεται μήνυμα αντίστοιχο με:

```
 * Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
```

Πρόσβαση στην εφαρμογή από τον browser

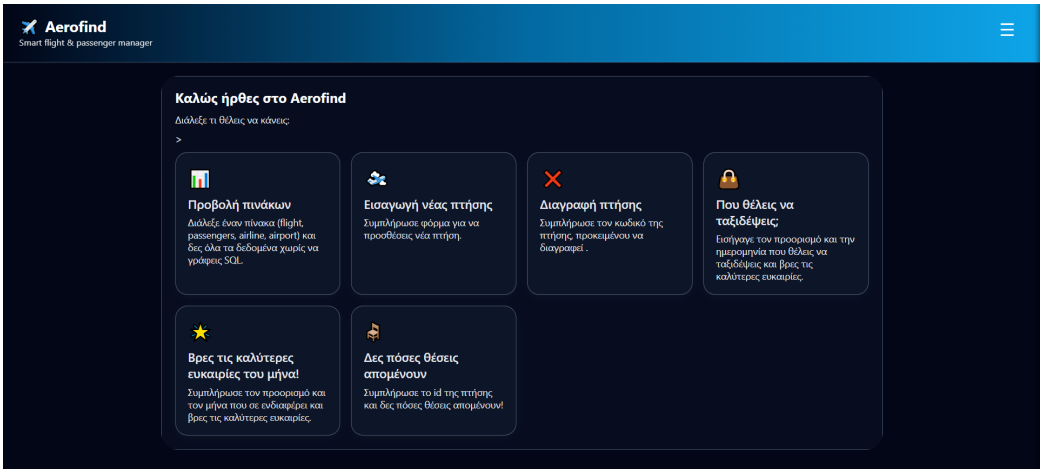
Για να ανοίξουμε τη σελίδα της εφαρμογής:

1. Κρατάμε πατημένο το πλήκτρο `Ctrl`.
2. Κάνουμε κλικ στο `http://127.0.0.1:5000` που εμφανίζεται στο `Command Prompt`, ή
3. Εναλλακτικά, γράφουμε τη διεύθυνση `http://127.0.0.1:5000` στη γραμμή διευθύνσεων του browser και πατάμε `Enter`.

Με αυτόν τον τρόπο ανοίγει η αρχική σελίδα της εφαρμογής `Aerofind` στον προεπιλεγμένο browser του συστήματος.

3.5 Οδηγίες περιήγησης στη σελίδα "Aerofind"

Από τη στιγμή που έχουν εκτελεστεί όλα τα βήματα της ενότητας 3.3 με επιτυχία, έχουμε μπει πλέον στην εφαρμογή **Aerofind**. Πρόκειται για ένα περιβάλλον ολοκληρωμένο ως προς το γραφικά και φιλικό προς τον χρήστη. Παρακάτω, φαίνεται η home page:



Σχήμα 7: Home page

Παρατηρούμε πως υπάρχουν 6 λειτουργίες. Επιπλέον, υπάρχει η μπάρα μενού πάνω δεξιά που καλύπτει τις έξι αυτές λειτουργίες.

Ας δούμε τι κάνει κάθε λειτουργία και πως να χρησιμοποιήσετε την κάθε μία. **Στο σημείο αυτό να τονίσουμε πως τα δεδομένα σε κάθε πίνακα έχουν προσαρμοστεί για τους σκοπούς της εργασίας. Για αυτό τον λόγο, και για να γίνει ευκολότερη και πιο κατανοητή η περιήγηση του χρήστη, υπάρχουν συγκεκριμένα βήματα που προτείνουμε να ακολουθεί ο χρήστης σε κάθε λειτουργία.**

1. **Προβολή πινάκων**

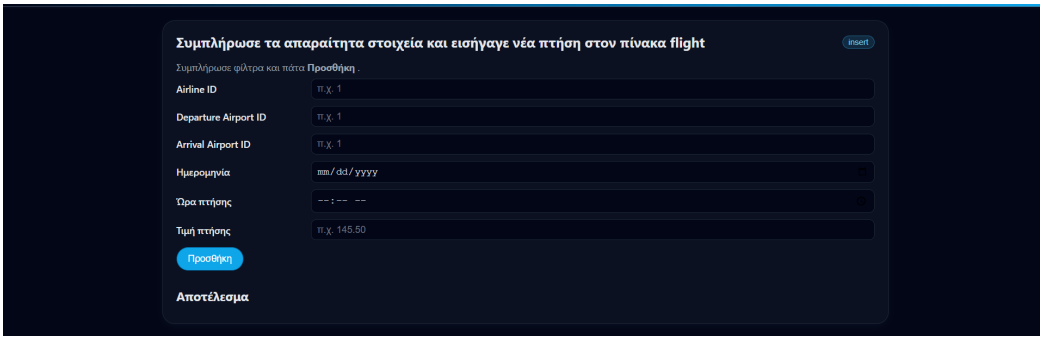
   Διαλέγοντας τον πίνακα που σε ενδιαφέρει, μπορείς να δεις όλα τα δεδομένα που έχουν εισαχθεί. **Θεωρούμε πιο σημαντικό και πρακτικό να γίνει αναζήτηση των πινάκων <<airline>> και <<airport>> σε αυτή την λειτουργία.**



Σχήμα 8: Λειτουργία 1: Προβολή πινάκων

2. **Εισαγωγή νέας πτήσης :**

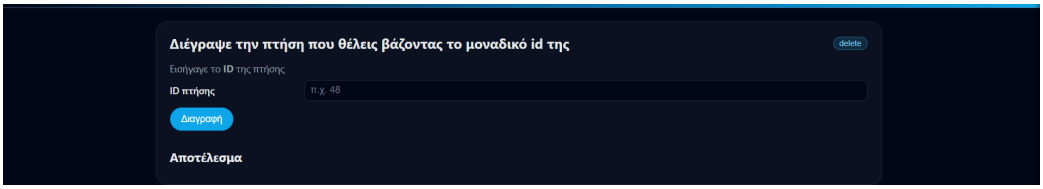
   Συμπληρώνοντας τα απαραίτητα στοιχεία, γίνεται αυτόματη πρόσθεση της νέας πτήσης στον πίνακα <<flight>>. Τα id μπορούν να βρεθούν από τους πίνακες <<airline>> και <<airport>>. Ενδεικτικό παράδειγμα: Airline ID = 1 (Aegean Airlines), Departure Airport ID = 1 (Athens International Airport), Arrival Airport ID = 4 (Frankfurt Airport) κτλ.



Σχήμα 9: Λειτουργία 2: Εισαγωγή νέας πτήσης

3. **Διαγραφή πτήσης :**

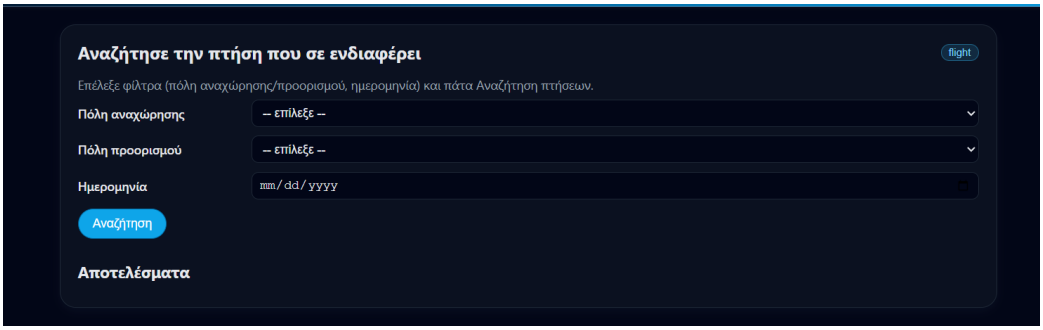
   Συμπληρώνοντας το μοναδικό id της πτήσης, η πτήση διαγράφεται αυτόματα.



Σχήμα 10: Λειτουργία 3: Διαγραφή πτήσης

4. **Που θέλεις να ταξιδέψεις :**

   Συμπληρώνοντας τα απαραίτητα στοιχεία, μπορείς να δεις την ημερομηνία, τιμή και ώρα για το ταξίδι που σε ενδιαφέρει. **Προτείνονται τα εξής: Πόλη αναχώρησης -> Παρίσι, Πόλη προορισμού -> Βαρκελώνη, Διανυκτερεύσεις -> 4, Ημερομηνία -> οποιαδήποτε τον Μάρτιο του 2026 πχ 03/12/2026 (κατά προτίμηση στα μέσα του Μαρτίου)**. Το αποτέλεσμα είναι η ημερομηνία που διάλεξες συνδυαστικά με δύο μέρες πριν και μετά από την ημερομηνία που εισήγαγες. Αυτό γίνεται ώστε ο χρήστης να μπορέσει να συγκρίνει τις τιμές εκείνη την περίοδο. Επιπλέον , εμφανίζονται αυτόματα και οι πτήσεις επιστροφής.



Σχήμα 11: Λειτουργία 4: Που θέλεις να ταξιδέψεις

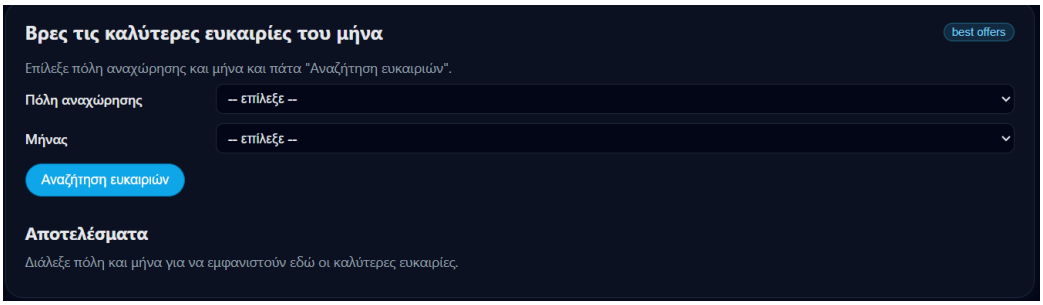
Παρακάτω το αποτέλεσμα.



Σχήμα 12: Λειτουργία 4: παράδειγμα

5. **Βρες τις καλύτερες ευκαιρίες του μήνα! :**

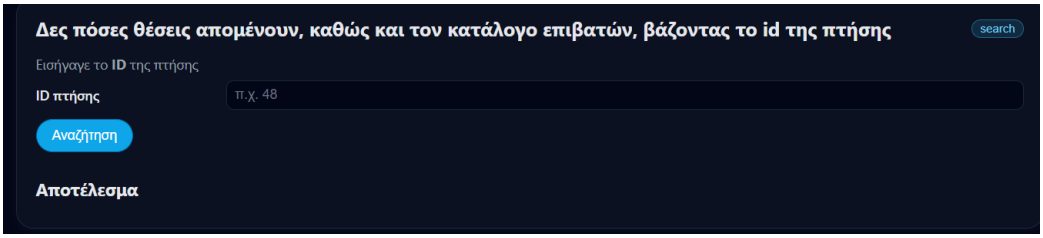
   Συμπλήρωσε την πόλη αναχώρησης και τον μήνα που σε ενδιαφέρει και δες όλες τις πτήσεις από την φθηνότερη προς την ακριβότερη. **Προτείνεται: Αθήνα και Απρίλιος.**



Σχήμα 13: Λειτουργία 5: Βρες τις καλύτερες ευκαιρίες του μήνα!

6. **Δες πόσες θέσεις απομένουν :**

   Συμπλήρωσε το id της πτήσης και δες το ονοματεπώνυμο όλων των επιβατών, τον αριθμό τους, το συνολικό αιρθμο θέσεων στο αεροσκάφος και τον υπολειπόμενο αριθμό θέσεων. **Προτεινόμενο id: 99.**



Σχήμα 14: Λειτουργία 6: Δες πόσες θέσεις απομένουν

4 Σχολιασμός κώδικα

4.1 Υλοποίηση Προβολής Πινάκων (View Tables)

Ο χρήστης μπορεί να επιλέξει έναν από τους πίνακες της βάσης δεδομένων μέσα από ένα dropdown με χρήση JavaScript. Το backend επιστρέφει όλες τις γραμμές του πίνακα σε μορφή JSON.

Ο σχετικός κώδικας Flask είναι:

```
@app.route("/view_table", methods=["GET"])
def view_table():
    table = request.args.get("table")

    if not table or table not in ALLOWED_TABLES:
        return jsonify({"status": "error",
                        "message": "Μη έγκυροόνομαπίνακα  ."})

    conn = get_connection()
    cursor = conn.cursor(dictionary=True)
    cursor.execute(f"SELECT * FROM {table}")
    rows = cursor.fetchall()
    conn.close()

    rows = normalize_rows(rows)
    return jsonify({
        "status": "success",
        "table": table,
        "rows": rows
    })
```

Η λειτουργία αυτή επιτρέπει πλήρη ορατότητα των καταχωρήσεων της βάσης χωρίς να απαιτείται SQL.

4.2 Εισαγωγή Νέας Πτήσης

Η εφαρμογή επιτρέπει την προσθήκη νέας πτήσης με όλα τα απαραίτητα πεδία: αεροπορική εταιρεία, αεροδρόμιο αναχώρησης και άφιξης, πόλεις, ημερομηνία, ώρα και τιμή.

Η λειτουργία ρυθμίζεται μέσω POST αιτήματος:

```
@app.route("/insert_flight", methods=["POST"])
def insert_flight():
    data = request.get_json()

```

```
    airline_id = int(data.get("airline_id"))
    dep_airport_id = int(data.get("departure_airport_id"))
    arr_airport_id = int(data.get("arrival_airport_id"))
    dep_city = data.get("departure_city")
    arr_city = data.get("arrival_city")
    flight_date = ...
    flight_time = ...

    cursor.execute("""
        INSERT INTO flight (...)
        VALUES (%s,%s,%s,%s,%s,%s,%s,%s)
    """)
```

Μετά την εισαγωγή, η εφαρμογή επιστρέφει την νέα πτήση σε JSON μορφή.

4.3 Διαγραφή Πτήσης

Ο χρήστης μπορεί να εισάγει ένα ID πτήσης και η εφαρμογή εκτελεί:

```
DELETE FROM flight WHERE id = %s
```

Η Flask διαδρομή είναι:

```
@app.route("/delete_flight", methods=["POST"])
def delete_flight():
    data = request.get_json()
    flight_id = data.get("id")
    cursor.execute("DELETE FROM flight WHERE id = %s", (
       flight_id,))
```

Επιστρέφεται μήνυμα επιτυχίας ή αποτυχίας.

4.4 Αναζήτηση Πτήσεων με Φίλτρα

Η αναζήτηση βασίζεται σε κριτήρια:

- πόλη αναχώρησης,
- πόλη προορισμού,
- διανυκτερεύσεις (π.χ. 4),
- ημερομηνία,
- εύρος ημερομηνιών (±2 ημέρες αυτόματα).

Η διαδρομή:

```
@app.route("/search_flights", methods=["GET"])
def search_flights():
    from_city = request.args.get("fromCity")
    to_city = request.args.get("toCity")
    total_days = request.args.get("days", type=int)
    date_str = request.args.get("flightDate")
```

Το σύστημα δημιουργεί δυναμικά το SQL query και επιστρέφει ταξινομημένες πτήσεις.

4.5 Αναζήτηση Επιβατών και Υπολογισμός Διαθέσιμων Θέσεων

Για κάθε πτήση, το σύστημα παρουσιάζει:

- όλους τους επιβάτες (πίνακας `passengers`),
- το capacity του αεροσκάφους,
- τον αριθμό επιβατών,
- τις διαθέσιμες θέσεις.

Ακολουθεί το σχετικό query:

```
SELECT ac.capacity
FROM aircraft ac
JOIN airline ai ON ac.airline_id = ai.airline_id
JOIN flight f ON f.airline_id = ai.airline_id
WHERE f.id = %s
```

και στη συνέχεια:

```
SELECT * FROM passengers WHERE flight_id = %s
```

Η Flask υλοποίηση:

```
@app.route("/search_passengers", methods=["GET"])
def search_passengers():
    flight_id = request.args.get("flight_id", type=int)
    ...
    remaining_seats = capacity - passenger_count
    return jsonify({...})
```

Με αυτόν τον τρόπο, ο χρήστης μπορεί να δει πλήρη κατάσταση πτήσης, πληρότητα και διαθέσιμες θέσεις.

5 Δεδομένα που χρησιμοποιήθηκαν

Στο κεφάλαιο αυτό παρουσιάζονται συνοπτικά τα δεδομένα που χρησιμοποιήθηκαν για τη δημιουργία και δοκιμή της εφαρμογής *Aerofind*. Τα δεδομένα είναι κατά κύριο λόγο **συνθετικά**, δημιουργήθηκαν δηλαδή αποκλειστικά για τους σκοπούς της εργασίας, ώστε:

- να υπάρχει επαρκής ποικιλία σε αεροδρόμια, αεροπορικές εταιρείες, πτήσεις και επιβάτες,
- να μπορούν να δοκιμαστούν ρεαλιστικά σενάρια αναζήτησης, εισαγωγής και διαγραφής δεδομένων,
- να μην χρησιμοποιούνται πραγματικά προσωπικά δεδομένα επιβατών.

Όλα τα αρχικά δεδομένα έχουν οργανωθεί σε `INSERT` statements μέσα στο αρχείο `code.sql`, το οποίο συνοδεύει την εφαρμογή και μπορεί να εκτελεστεί ως σύνολο για την πλήρη αναδόμηση της βάσης.

5.1 Είδη δεδομένων ανά πίνακα

Για κάθε πίνακα της βάσης δεδομένων `uni_app` χρησιμοποιήθηκαν τα εξής είδη δεδομένων:

- **airport**: περιλαμβάνει ονομασίες αεροδρομίων, πόλεις και χώρες από πολλές περιοχές του κόσμου, ώστε να υποστηρίζονται διεθνείς πτήσεις (π.χ. Athens International Airport, Heathrow Airport, Charles de Gaulle Airport κ.ά.).
- **airline**: περιλαμβάνει πραγματικά ονόματα αεροπορικών εταιρειών (π.χ. Aegean Airlines, British Airways, Lufthansa, Emirates κ.λπ.), με τη χώρα προέλευσής τους.
- **aircraft**: για κάθε αεροπορική έχει εισαχθεί ένα αεροσκάφος, με χωρητικότητα (`capacity`), κατασκευαστή, μοντέλο, έτος κατασκευής και κατάσταση λειτουργίας (`status` = active / stored / retired).
- **flight**: κάθε εγγραφή αντιστοιχεί σε συγκεκριμένη πτήση και συνδέεται, μέσω ξένων κλειδιών, με την αεροπορική εταιρεία και τα αεροδρόμια αναχώρησης/άφιξης. Για κάθε πτήση αποθηκεύονται επίσης η ημερομηνία , η ώρα και η τιμή .

- **passengers**: περιέχει συνθετικά ονόματα επιβατών (ελληνικά και ξένα, σε λατινικούς χαρακτήρες), μαζί με φύλο και τηλέφωνο επικοινωνίας. Κάθε επιβάτης συνδέεται με μία συγκεκριμένη πτήση μέσω του `flight_id`.

5.2 Ποσότητα και κάλυψη δεδομένων

Ο συνολικός όγκος δεδομένων έχει σχεδιαστεί έτσι ώστε:

- να υπάρχουν **πολλά διαφορετικά αεροδρόμια** (δεκάδες πόλεις και χώρες), ώστε τα φίλτρα αναζήτησης να έχουν νόημα,
- να καλύπτονται **πολλές αεροπορικές εταιρείες**, τόσο παραδοσιακές όσο και low–cost,
- να υπάρχουν **εκατοντάδες πτήσεις** με διαφορετικούς συνδυασμούς προορισμών, ημερομηνιών και τιμών,
- να υπάρχουν **πολλοί επιβάτες ανά πτήση**, ώστε να μπορεί να ελεγχθεί ο υπολογισμός πληρότητας και εναπομεινάντων θέσεων.

Ενδεικτικά, ο πίνακας 2 συνοψίζει το πλήθος εγγραφών ανά πίνακα, όπως αυτό προέκυψε στην τελική έκδοση της εργασίας.

Πίνακας 2: Πλήθος εγγραφών ανά πίνακα της βάσης

Πίνακας	**Ενδεικτικό πλήθος εγγραφών**
`airport`	$\approx$ 150 αεροδρόμια
`airline`	35 αεροπορικές εταιρείες
`aircraft`	25 αεροσκάφη
`flight`	$\approx$ 40000 πτήσεις
`passengers`	αρκετές χιλιάδες επιβάτες

Τα μεγέθη αυτά κρίνονται επαρκή για να δοκιμαστούν:

- τα ερωτήματα προβολής πίνακα,
- τα φίλτρα αναζήτησης πτήσεων,
- ο μηχανισμός εύρεσης καλύτερων προσφορών,
- ο υπολογισμός πληρότητας πτήσης και υπολειπόμενων θέσεων.

5.3 Περιορισμοί και παραδοχές

Οι τιμές που χρησιμοποιούνται (τιμές εισιτηρίων, χωρητικότητες αεροσκαφών, ημερομηνίες, ονόματα επιβατών) είναι **ενδεικτικές** και δεν αντιστοιχούν σε πραγματικά δρομολόγια ή πραγματικούς πελάτες. Ο σκοπός τους είναι αποκλειστικά η επίδειξη:

- της σωστής λειτουργίας των περιορισμών ακεραιότητας (πρωτεύοντα/ξένα κλειδιά, `CHECK` constraints),
- της ορθής λειτουργίας των SQL ερωτημάτων που υλοποιούνται μέσα από την εφαρμογή,
- της αλληλεπίδρασης του front-end με τη βάση δεδομένων.

Για μελλοντική επέκταση, η ίδια δομή της βάσης μπορεί να λειτουργήσει με πραγματικά δεδομένα πτήσεων, αρκεί να αντικατασταθούν τα συνθετικά `INSERT` statements από πραγματικές εγγραφές.

5.4 Αυτοματοποιημένη Εισαγωγή Δεδομένων με Python Scripts

Για την επιτάχυνση της διαδικασίας δημιουργίας και συμπλήρωσης της βάσης δεδομένων, χρησιμοποιήθηκαν δύο βοηθητικά Python scripts, τα οποία επέτρεψαν την αυτόματη παραγωγή και εισαγωγή μεγάλων όγκων δεδομένων για τους πίνακες `flight` και `passengers`. Η χρήση των scripts ήταν απαραίτητη, καθώς η χειροκίνητη καταχώρηση εκατοντάδων πτήσεων και χιλιάδων επιβατών θα ήταν ιδιαίτερα χρονοβόρα και επιρρεπής σε λάθη.

Εισαγωγή Πτήσεων

Το πρώτο script δημιούργησε συνδυασμούς πτήσεων μεταξύ των αεροδρομίων που υπήρχαν στη βάση, λαμβάνοντας υπόψη:

- την αντιστοίχιση `airline` → `aircraft`,
- ρεαλιστικές ημερομηνίες και ώρες αναχώρησης,
- εύρος τιμών εισιτηρίων βασισμένο στην απόσταση των προορισμών,
- αποφυγή διπλοεγγραφών (ίδια πτήση ίδια ημέρα).

Το script παρήγαγε αυτόματα `INSERT` statements σε μορφή συμβατή με MySQL, τα οποία στη συνέχεια εκτελέστηκαν μαζικά για την πλήρωση του πίνακα `flight`.

Εισαγωγή Επιβατών

Το δεύτερο script χρησιμοποιήθηκε για τη δημιουργία μεγάλου συνόλου επιβατών ανά πτήση. Για κάθε `flight_id`:

- δημιουργήθηκαν τυχαία ονόματα (ελληνικά και ξένα, σε λατινικούς χαρακτήρες),
- αποδόθηκε φύλο και τηλέφωνο,
- ελέγχθηκε ότι ο αριθμός επιβατών δεν υπερβαίνει τη χωρητικότητα του αντίστοιχου αεροσκάφους,
- παράχθηκαν αυτόματα τα αντίστοιχα `INSERT` statements.

Με τον τρόπο αυτό, ο πίνακας `passengers` απέκτησε ρεαλιστική κατανομή επιβατών, επιτρέποντας την πλήρη λειτουργική δοκιμή των:

- αναζητήσεων επιβατών,
- υπολογισμού πληρότητας πτήσης,
- ανάκτησης υπολειπόμενων θέσεων μέσω του REST endpoint της εφαρμογής.

Πλεονεκτήματα της Χρήσης Scripts

Η χρήση των Python scripts προσέφερε σημαντικά οφέλη:

- **ταχύτητα**: η συμπλήρωση της βάσης ολοκληρώθηκε σε λίγα δευτερόλεπτα,
- **ακρίβεια**: αποφεύχθηκαν χειροκίνητα λάθη πληκτρολόγησης,
- **επεκτασιμότητα**: μπορούν να δημιουργηθούν νέες πτήσεις/επιβάτες οποιαδήποτε στιγμή με μικρές τροποποιήσεις,
- **ρεαλισμός**: οι επιβάτες και οι πτήσεις κατανέμονται με τρόπο που προσομοιώνει μια πραγματική βάση συστήματος κρατήσεων.

Τα scripts αυτά συνοδεύουν την εργασία και παρέχονται στον φάκελο παραδοτέων, ώστε να είναι δυνατή η πλήρης αναπαραγωγή της βάσης δεδομένων.

6 Συμπεράσματα και μελλοντική χρήση

Η παρούσα εργασία είχε ως στόχο την ανάπτυξη μιας ολοκληρωμένης εφαρμογής διαχείρισης αεροπορικών ταξιδιών, η οποία να επιτρέπει την εύχρηστη αλληλεπίδραση με μια σχεσιακή βάση δεδομένων μέσω ενός φιλικού frontend περιβάλλοντος. Ο μέγιστος σκοπός της προγραμματιστικής εργασίας ήταν να καταφέρουμε να σχεδιάσουμε μία ολοκληρωμένη βάση δεδομένων (χρήση MySQL), να υλοποιήσουμε σύνθετα ερωτήματα SQL, να δημιουργήσουμε ένα φιλικό περιβάλλον ως προς τον χρήστη (HTML, CSS, Javascript) και να διασυνδέσουμε το frontend με το backend χρησιμοποιώντας τη γλώσσα Python και τις βιβλιοθήκες Flask και mysql.connector.

Κατά την ανάπτυξη της εφαρμογής σχεδιάστηκαν έξι (6) λειτουργίες. Οι πρώτες τρεις (3) λειτουργίες ("Προβολή πινάκων", "Εισαγωγή πτήσεων", "Διαγραφή πτήσεων") υλοποιήθηκαν κυρίως για τους εκπαιδευτικούς σκοπούς της εργασίας και για να εξετάσουμε πως αλλάζουν δυναμικά τα δεδομένα της βάσης μας. Οι επόμενες τρεις λειτουργίες ("Που θέλεις να ταξιδέψεις;", "Ευκαιρίες του μήνα", "Υπολειπόμενες θέσεις") προσομοιώνουν πιο ρεαλιστικές συνθήκες και επιτρέπουν στον χρήστη να αλληλεπιδράσει με την εφαρμογή με βάση πραγματικά ζητήματα που συναντά κανείς σε ένα σύστημα διαχείρισης αεροπορικών ταξιδιών.

Η εφαρμογή μπορεί εύκολα να επεκταθεί και να αποτελέσει βάση για πιο ολοκληρωμένο σύστημα κρατήσεων. Αρχικά, μπορούμε να γεμίσουμε τους πίνακες με τόσα δεδομένα ώστε να καλύπτεται κάθε δυνατή πτήση, καθώς μέχρι στιγμής τα δεδομένα έχουν προσαρμοστεί με τέτοιο τρόπο ώστε να εκτελούνται συγκεκριμένες λειτουργίες. Επιπλέον, μπορούμε να προσθέσουμε σύστημα κράτησης θέσεων. Μπορούμε, επίσης, να δημιουργήσουμε "Login" για διαχειριστές και χρήστες. Τέλος, η ενσωμάτωση αληθινών APIs και η αναζήτηση πραγματικών πτήσεων από τρίτα συστήματα μπορεί να δημιουργήσει μία άκρως ρεαλιστική εφαρμογή διαχείρισης αεροπορικών ταξιδιών.